KING SAUD UNIVERSITY

College of Computer and Information Sciences

Ride-Sharing System

CSC 212 | Phase 1 Project Report | Fall 2026

Team member	Student ID	Responsibility
Osamah Alsheddi	443102410	A - Data structures
Ibrahim Mohammed Aljebreen	445100804	B - System logic and CSV
Mohammed Alsamari	445100816	C - Models, menu and report

Purpose

Implement a Java ride-sharing application using custom linked lists, conflict-free scheduling, searches, and consistent removal of related records.

The responsibility split is retained for team review and demonstration. All team members should understand the complete application flow.

1. Design and system behavior

Three cooperating layers

Main reads input and prints results. RideSharingSystem validates operations and coordinates the lists. Model classes hold the people, vehicles, dates and rides. Supplied interface signatures are unchanged.

Layer	Classes and responsibilities
A: custom structures	LinkedList, RiderList, DriverList and RideList. Insert, search, remove and ordered retrieval.
B: coordination	RideSharingSystem. Uniqueness, existence checks, scheduling, histories, cascade removal and CSV loading.
C: models and menu	Person, Rider, Driver, VehicleType, DateTime, Ride, PrivateRide, SharedRide and Main. Report assembly.

Ordering without expensive listing

RiderList and DriverList each keep two custom linked lists of references to the same objects: one sorted by ID and one sorted by full name. This meets the ID-storage rule and supports alphabetical getAll() in O(n). Both indexes are updated on insertion and removal. RideList is stored alphabetically by pickup location.

LinkedList keeps head, tail and size. The tail makes append O(1). Its custom Cursor visits each node once and can unlink the last returned node in O(1). No java.util collection is used. Repeated get(i) on a linked list is avoided in application traversals.

Record and time rules

Person ID is final; DateTime fields and ride pickup/dropoff references are final. Riders and drivers compare by ID. DateTime compares year, month, day, hour, then minute. Ride compares pickup location. Phone numbers require exactly 10 digits; plates require three uppercase letters followed by four digits.

Rider IDs and emails are unique. Driver IDs and plates are unique. Shared rides require two or more distinct riders at creation, but may have one participant after a removal. String searches are exact and case-sensitive.

Mutation boundary

All normal application changes go through RideSharingSystem. Retrieval copies list nodes, not model objects. Required model setters remain public; direct external edits to indexed fields or histories are outside the menu workflow and can break invariants. An edit feature would need revalidation and reindexing.

2. Worst-case analysis: notation

Symbol	Meaning
N, D, R	Current numbers of riders, drivers and rides.
K	Number of riders requested for a new shared ride.
M	Maximum participants on any existing shared ride; at least 1 in bounds.
H	Maximum length of a person's ride history, at most R under normal system use.
P	Total participant references across the shared rides matched by a query.
Q, L	Number of CSV data rows and total number of input characters respectively.

Record field lengths are treated as bounded for the tables below. If string lengths are unbounded, multiply string comparison/validation costs by their lengths and include produced output length for toString and console display. CSV analysis explicitly includes total characters L.

For a full traversal of n nodes, T(n) = a*n + b, so T(n) is O(n). Two separate traversals cost a*n + b*n + c, still O(n). Nested traversals multiply; a triangular duplicate scan sums 0 + 1 + ... + (K-1) = K(K-1)/2, which is O(K^2).

LinkedList<T>: all public operations

Method	Growth-rate reasoning	Worst case
Constructor; size(); cursor()	Initialize/read a fixed number of fields.	O(1)
add(item)	Create one node and update tail/head and size.	O(1)
add(index,item)	Walk at most n nodes, then change links.	O(n)
get(index)	Walk from head to index; at most n nodes.	O(n)
remove(index)	Walk at most n nodes, then cursor removal.	O(n)
copy()	n cursor steps and n constant-time appends.	O(n)
Cursor.hasNext/next/remove	Each reads or changes a fixed number of references.	O(1)

Auxiliary space: add creates one node; copy creates n nodes. Cursor uses a fixed set of node references. Invalid indices are rejected, and removing the last node updates the tail.

3. People-list method costs

Use n = N for RiderList and n = D for DriverList. ID and name indexes each contain n references. They do not duplicate Person objects.

Methods	Growth-rate reasoning	Worst case
RiderList/DriverList constructors	Create two empty custom lists.	O(1)
add(person)	ID lookup + email/plate lookup + ID-position scan/insertion + name-position scan/insertion; at most c*n operations.	O(n)
findById	Traverse ID list; may stop early but may visit all n nodes.	O(n)
RiderList.findByEmail; DriverList.findByVehiclePlate	One full scan at worst.	O(n)
findByName	One scan, append each match in O(1).	O(n)
RiderList.findByHomeCity; DriverList.findByVehicleType	One scan, append every matching record.	O(n)
getAll	Copy the name-ordered index in one pass.	O(n)
removeById	Scan ID storage, unlink, then scan/unlink name index. n+n bounds the work.	O(n)
RiderList.removeByEmail; DriverList.removeByVehiclePlate	Scan/unlink one record, then remove its ID from name index.	O(n)
removeByName	Scan and remove every match in ID storage, then every match in the name index. n+n steps.	O(n)
RiderList.removeByHomeCity; DriverList.removeByVehicleType	Same two complete scans; cursor removal is O(1) per match.	O(n)
size	Read stored list size.	O(1)
Private removeNameIndex	At most n cursor steps. Used by single-record removals.	O(n)

Search results and getAll allocate at most n result nodes. Mutating a returned list does not remove records from storage. Bulk removals continue after a match; they do not stop after the first duplicate name, city or vehicle type.

List classes handle storage operations only. To remove a person with ride cleanup, Main or other callers must use RideSharingSystem.removeRider/removeDriver rather than calling a list removal directly.

4. Ride-list and model costs

RideList method	Growth-rate reasoning	Worst case
Constructor; size	Initialize/read stored size.	O(1)
addRide	R-node scan for ID uniqueness and pickup position, then at most R links to insert.	O(R)
removeRideById	Scan until matching ride ID, at most R nodes.	O(R)
getAllAlphabetically	Copy already ordered nodes; no sorting on retrieval.	O(R)
findByPickupLocation	R-node scan plus constant append for each match.	O(R)
findByRiderName	Visit each ride; private check O(1), shared participant copy and scan O(M). Each ride is appended at most once.	O(R*M)

Model methods	Growth-rate reasoning	Worst case
Person constructor; getId/getName/getPhoneNumber/getRideHistory; setName/setPhoneNumber	Fixed fields; phone format is fixed length. History getter returns a reference, not a copy.	O(1)
Rider/Driver constructors; getters, setters and compareTo	Fixed fields; plate check is fixed length; compareTo compares IDs.	O(1)
DateTime constructor, getters, format, compareTo, parse	Fixed five fields; fixed-format input; constant-size month table and leap check.	O(1)
Ride constructor, getters, setters, compareTo	Fixed fields; bounded pickup string comparison.	O(1)
PrivateRide constructor; getRider/setRider/hasRider	One rider reference and one ID comparison.	O(1)
SharedRide constructor; isEmpty	Create empty list / read its size.	O(1)
SharedRide.getParticipants	Copy k participant nodes.	O(k)
SharedRide.hasRider/removeParticipantById/addParticipant	At most k nodes for membership/removal; append O(1).	O(k)
Person subclasses and PrivateRide.toString	Bounded fields and fixed-format time values.	O(1)
SharedRide.toString	One pass over k participants and their bounded details.	O(k)

VehicleType is an enum of SEDAN, LUXURY_SEDAN, SUV and VAN. Its ordinary value lookup is constant for this fixed set. Constructors and inherited methods are covered above. Tests and launch scripts are support tools, not application data-structure operations.

5. System searches and additions

RideSharingSystem methods	Growth-rate reasoning	Worst case
Constructor	Create RiderList, DriverList, RideList.	O(1)
addRider	Validation + uniqueness scans + RiderList.add. Constant number of N-node scans.	O(N)
addDriver	Validation + ID/plate scans + DriverList.add. Constant number of D-node scans.	O(D)
searchRiderById; searchRiderByEmail	Delegate to one rider lookup.	O(N)
searchRidersByName; searchRidersByHomeCity; getAllRiders	Delegate to one full scan or a copy.	O(N)
searchDriverById; searchDriverByVehiclePlate	Delegate to one driver lookup.	O(D)
searchDriversByVehicleType; getAllDrivers	Delegate to one full scan or a copy.	O(D)
searchRidesByPickupLocation; getAllRidesAlphabetically	Delegate to scan / ordered copy.	O(R)
searchRidesByRiderName	Delegate to ride/participant scan.	O(R*M)
getSharedRideParticipants	Find matching rides O(R); copy/visit P people; each distinctness check scans up to P result nodes, sum <= P^2.	O(R + P^2)

Private scheduling

schedulePrivateRide first validates the locations and time range, finds one rider and one driver, then scans their histories. Only after all checks succeed does it create a PrivateRide, insert it into the ordered ride list and append the same ride reference to both histories.

T = c + a*N + b*D + c1*Hrider + c2*Hdriver + d*R. Therefore the worst-case cost is O(N + D + H + R). New history nodes and the ride object take O(1) additional space.

Shared scheduling

scheduleSharedRide checks the driver and every requested rider. Repeated IDs are rejected by a triangular scan. K rider lookups cost O(K*N); K history checks cost O(K*H). Adding participants performs membership checks whose total is O(K^2). Ordered ride insertion costs O(R).

T = O(D + H + K*N + K*H + K^2 + R). Extra space is O(K) for the participant array/list and history nodes. No system list or history changes before validation completes.

6. Conflicts and cascade removal

The overlap rule

Two half-open intervals [p1,d1) and [p2,d2) overlap exactly when p1 < d2 AND p2 < d1. DateTime.compareTo provides each comparison. Equality at a boundary is allowed: 09:00-10:00 and 10:00-11:00 do not overlap. Pickup must be strictly earlier than dropoff.

Helper	Growth-rate reasoning	Worst case
overlaps; hasText; validTime; validRide	Fixed comparisons, date components or bounded fields.	O(1)
isAvailable	Scan up to H history nodes, overlap test per node.	O(H)
removeFromHistory	Scan H nodes and unlink matches with cursor.	O(H)
deleteRide	Find/remove ride O(R), clean driver history O(H), then one rider or up to M histories; copy shared participants O(M).	O(R + H + M*H + M)

removeRider(riderId)

Find the rider, then traverse a snapshot of all rides. Delete each of their private rides from storage and histories. For each shared ride, remove only that rider and their history entry; delete the shared ride and driver history entry if it becomes empty. Finally remove the rider from both rider indexes.

R visits may each inspect M participants and clean a history of H nodes. In the worst case up to R private rides are deleted, each scanning the R-node ride list. A safe bound is T = O(N + R*(R + H + M)). The snapshot uses O(R) auxiliary nodes.

removeDriver(driverId)

Find the driver, then traverse a snapshot of all rides. For every assigned ride, delete it and clean the driver and all participant histories. Finally remove the driver from both indexes. No shared ride can remain without its assigned driver.

At most R assigned rides are deleted. Each deletion costs O(R + H + M*H + M). Thus T = O(D + R*(R + H + M*H + M)). Additional snapshot/participant-copy space is O(R + M). These conservative bounds include the actual linked-list searches.

Whole-system storage

Let E be the sum of rider participation counts across all rides. Storage is O(N + D + R + E): people and their two indexes, rides and driver-history references, participant lists and rider-history references. Name indexes duplicate references, not the model objects.

7. CSV loading and menu costs

CSV behavior

loadRidersFromCSV, loadDriversFromCSV and loadRidesFromCSV call loadCSV with a file type. It accepts optional headers, a leading UTF-8 BOM, blank lines, quoted fields and semicolon-separated shared IDs. Each record is one line. It checks the expected field count and uses the same add/schedule methods as the menu.

Riders load before drivers and rides. A ride must reference people already loaded. Invalid rows do not stop later valid rows. Valid rows are retained, but the final boolean is false if any row is rejected or reading fails; there is no whole-file rollback.

Method	Growth-rate reasoning	Worst case
splitCSV(line)	Each character/field processed once; constant-time list append and cursor-to-array conversion.	O(line length)
parseDateTime	Delegate to fixed-format DateTime.parse.	O(1)
loadRow	Parse fields/IDs, then add one person or schedule one ride.	O(row length + operation cost)
loadCSV	Read L characters plus the sum of all Q row operation costs.	O(L + sum of row costs)
loadRidersFromCSV	As rider count grows from N0, sum over rows is Q*N0 + Q*(Q-1)/2.	O(L + Q*N0 + Q^2)
loadDriversFromCSV	Same sum with initial driver count D0.	O(L + Q*D0 + Q^2)
loadRidesFromCSV	Sum schedule costs as rides/histories grow; use maximum K,H,R during load.	O(L + Q*(D + H + K*N + K*H + K^2 + R))

The bound for ride loading covers mixed private/shared rows, treating K as at least 1. Current R and H in that bound include rides accepted during loading. Working memory is the largest row plus its fields/IDs; accepted objects become system storage.

Main: all menu and helper operations

Method	Cost excluding console/device latency
menu; outcome; loadResult	Constant number of fixed messages: O(1).
read; number	O(length of entered text); bounded ID conversion is O(1).
display(list)	One custom cursor step per result: O(n) for people/private rides; O(n + participant details printed) for shared rides.
addRider; addDriver	Input + model construction + O(N) / O(D) system addition.
addRide	Input plus private/shared scheduling cost described in Section 5.
main	Startup costs are the three CSV loads. Each iteration costs input + selected method + display. Across t choices, sum these t iteration costs; there is no fixed bound without t.

8. Validation and use

Executed checks

The application compiled with Microsoft OpenJDK 21 using Java 8 target compatibility and no warnings under the selected lint checks. The integration suite uses the real linked-list and system classes. It completed normally; failed checks would throw AssertionError.

Area	Verified behavior
Custom structures	Insertion, bounds, cursor removal, tail reset, sorted views, duplicate ID/email/plate rejection and complete multi-match removals.
Models and dates	Phone/plate validation, leap days, invalid dates/hours, chronological comparison and ID-based comparison.
Scheduling	Missing records, reversed/zero duration, overlapping/equal/containing intervals, touching boundaries, shared duplicates and no partial changes.
Cascade removal	Private deletion, surviving shared ride, empty shared deletion, driver deletion, cleaned histories and reuse of freed time.
CSV	Headers, BOM, quoted comma, both ride types, invalid rows, invalid references, missing files and valid-row retention.
Menu	All 12 options exercised; rider/driver/private/shared additions succeeded; an overlapping ride was rejected; invalid choices handled.

Dataset limitation

The official riders_100.csv, drivers_30.csv and rides_40.csv files were not present in the supplied download. Validation used labelled synthetic examples and temporary test CSVs. Official-data loading has not been verified. Default Main startup attempts the official filenames; --demo explicitly chooses the synthetic examples.

Run and handover

From the project folder, compile with javac -encoding UTF-8 -d build *.java. Run java -cp build Main after adding the three official CSV files. Use java -cp build Main --demo to rehearse with the included examples. test.bat compiles and runs the integration checks.

Main contains exactly the required numbered options 1 through 12 and 0 to exit. The specification does not include a removal option, so removal behavior is checked through the public system methods and integration suite.

Team review

Osamah explains node links, ordered insertion and list complexity. Ibrahim explains scheduling, overlap checks, histories, cascades and CSV calls. Mohammed explains inheritance, interfaces, immutable time fields, input handling and report organization. Every member should trace a ride from menu choice to the stored object and explain its complexity.

Requirement reference: supplied CSC 212 Ride-Sharing System Phase 1 assignment, Fall 2026. The original responsibility split and team identities were supplied by the team.